

API Test Report

Practice Software Testing API

Products endpoint test coverage and findings

Submitted by

Marko Djordjevic

ESA Gaming - QA Engineer Test Task

1. Executive Summary

This report documents the API test execution against the Practice Software Testing public API (<https://api.practicesoftwaretesting.com>), focusing on the POST /products endpoint as specified in the assignment.

A Postman collection containing 11 requests with 29 assertions was developed, covering authentication setup, security boundary checks, input validation, and a positive read-path. The collection runs both interactively in Postman and from the command line via Newman.

Test execution results:

Metric	Value
Total requests executed	11
Total assertions	29
Assertions passed	23
Assertions failed	6
Pass rate	79%
Total run duration	2.8 s
Average response time	171 ms
API bugs discovered	4

Six failing assertions are not test defects - they are intentional. Each test was written against the expected correct behavior of the API. When the API deviates from the specification (for example, accepting a POST without authentication), the test fails by design and documents the defect. See Section 4 (Findings) and the accompanying Bug_Report.pdf for full details.

2. Test Scope

2.1 In scope

- POST /products - product creation endpoint (primary target)
- POST /users/login - authentication setup
- GET /categories/tree, GET /brands, GET /images - prerequisite setup endpoints
- GET /products - positive read-path verification

2.2 Out of scope

- PUT/PATCH and DELETE on /products (not required by the assignment)
- Performance and load testing
- Frontend / UI testing (covered separately by the Cypress suite)

- Penetration testing (SQL injection, XSS, etc.)

2.3 Test environment

Property	Value
Base URL	https://api.practicesoftwaretesting.com
API documentation	https://api.practicesoftwaretesting.com/api/documentation
Tools used	Postman 12.x, Newman 6.2.2, newman-reporter-htmlextra
Auth method	Bearer token (Sanctum)
Test user role	Customer

3. Test Cases

All 11 test cases below were implemented as Postman requests with one or more assertions in the test scripts. The Postman collection is available in `postman/ESA_Products_API_Tests.postman_collection.json`.

#	Test case	Method	Type	Expected	Result
01	Login - valid credentials	POST /users/login	Setup	200 OK + token	PASS
02	Get Categories	GET /categories/tree	Setup	200 OK + array	PASS
03	Get Brands	GET /brands	Setup	200 OK + array	PASS
04	Get Product Images	GET /images	Setup	200 OK + array	PASS
05	Customer role not authorized to create product	POST /products	Security	401 / 403	FAIL (bug)
06	POST without auth token	POST /products	Security	401 Unauthorized	FAIL (bug)
07	POST with invalid bearer token	POST /products	Security	401 Unauthorized	FAIL (bug)
08	POST missing required field: name	POST /products	Validation	422 + error	PASS
09	POST missing required field: product_image_id	POST /products	Validation	422 + error	PASS
10	POST with negative price (-10.00)	POST /products	Validation	422 + error	FAIL (bug)
11	Get all products	GET /products	Positive	200 OK + data[]	PASS

Setup tests (01-04) populate environment variables (accessToken, categoryId, brandId, productId) which are then consumed by the dependent POST /products tests via {{variable}} syntax.

4. Findings

Four defects were identified during test execution. Three of them are security-relevant (authentication and authorization), one is a business-rule input validation gap.

Bug ID	Summary	Expected	Actual	Severity
BUG-01	POST /products without auth token creates a product	401	201	High (security)
BUG-02	POST /products with an invalid bearer token creates a product	401	201	High (security)
BUG-03	POST /products accepts negative price value (-10.00)	422	201	Medium (business rule)
BUG-04	Customer role authorization checked after input validation	403 first	422 (validation runs first)	Low (ordering)

Full reproduction steps, request/response samples and recommended fixes are documented in docs/Bug_Report.pdf.

5. Test Execution

5.1 Running from Postman GUI

Open the collection 'Products API Tests' in Postman, select the 'Practice API - Prod' environment, then either:

- Run individual requests in order (01 - 11) by clicking Send on each
- Click the collection 'Run' button (Collection Runner) to execute all 11 requests in sequence

5.2 Running from the command line (Newman)

Prerequisites: Node.js 18 or higher.

Install Newman and the HTML reporter:

```
npm install -g newman
```

```
npm install -g newman-reporter-htmlextra
```

Run the collection with the HTML report enabled:

```
newman run postman/ESA_Products_API_Tests.postman_collection.json -e
postman/ESA_Practice_API.postman_environment.json -r "cli,htmlextra" --reporter-
htmlextra-export docs/newman-report.html
```

The HTML report is written to docs/newman-report.html and can be opened in any browser.

6. Conclusion

The POST /products endpoint was tested across 11 cases covering authentication, authorization, input validation, and the positive read-path. The Newman summary (23 / 29 assertions passing, 4 documented bugs) demonstrates that:

- Setup and read endpoints (login, categories, brands, images, GET products) work as specified.
- Required-field validation (name, product_image_id) works correctly.
- Authentication is not enforced on POST /products - tokens are not required and invalid tokens are accepted. This is the most serious finding.
- Business-rule validation on the price field is missing - negative values are persisted.
- Authorization ordering is suboptimal - input validation runs before the role check, leaking field-level information to unauthorized callers.

Recommendations:

- Prioritize fixes for BUG-01 and BUG-02 before any production deployment.
- Add server-side validation for price > 0 and any other business invariants.
- Re-run the Postman collection after each fix and confirm the corresponding assertion turns green.

Once the four bugs are fixed, the collection should report 29 / 29 passing assertions without modification - the assertions are already written against the correct expected behavior.